# Education Consultancy CRM: API Contract (v1)

REST API between the React client and the Node/Express server. Pairs with `schema.sql` (PostgreSQL). Frontend can build against these shapes with mock data while the backend catches up.

## 1. Conventions

- **Base URL:** `/api/v1`. JSON in and out, except file uploads (`multipart/form-data`).
- **Naming:** JSON uses `camelCase`; the database uses `snake_case`. Dates are ISO 8601 strings (`2026-11-05` or `2026-11-05T09:30:00Z`). IDs are integers.
- **Auth:** `Authorization: Bearer <jwt>` on every route except `POST /auth/login`. Token lifetime 8 hours.
- **Permissions:** every route lists the permission it needs as `module:action`. The server loads the user's permissions (with scope) from the role matrix. Scope `all` means any record; scope `own` means records assigned to or created by the user.
- **Out-of-scope records:** a missing permission returns `403`. A record outside the user's `own` scope returns `404`, so IDs don't leak.
- **Pagination:** list endpoints accept `?page=1&limit=20` (max 100) and return `{ "data": [...], "meta": { "page": 1, "limit": 20, "total": 134, "totalPages": 7 } }`.
- **Sorting:** `?sort=-createdAt` (prefix `-` for descending). Each list documents its allowed sort fields.
- **Search:** `?q=` does a case-insensitive match on the fields listed per endpoint.
- **Single resources** return the object directly, without a wrapper.
- **Deletes** return `204 No Content`. Catalog items are deactivated (`isActive: false`) instead of deleted.

### Error format

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request",
    "details": [{ "field": "email", "message": "Must be a valid email" }]
  }
}
```

| HTTP | `code` | When |
|---|---|---|
| 400 | `VALIDATION_ERROR` | Body or query fails validation |
| 401 | `UNAUTHENTICATED` | Missing, invalid, or expired token; bad login |
| 403 | `FORBIDDEN` | Role lacks the permission |
| 404 | `NOT_FOUND` | Missing record, or outside the user's scope |
| 409 | `CONFLICT` | Duplicate (email, application, checklist item) or invalid state transition |
| 413 | `FILE_TOO_LARGE` | Upload over 10 MB |
| 415 | `UNSUPPORTED_FILE_TYPE` | Upload not PDF, JPG, PNG or DOCX |
| 500 | `INTERNAL_ERROR` | Unexpected server error |

## 2. Permission matrix

The seed data in `schema.sql` implements this. A dash means no access. `own` / `all` is the scope.

| Module | Admin | Counselor | Front desk |
|---|---|---|---|
| users | all (CRUD) | - | - |
| leads | all (CRUD) | view, create, update: own | view, create, update: own |

| Module | Admin | Counselor | Front desk |
|---|---|---|---|
| tasks | all (CRUD) | view, create, update, delete: own | view, create, update: own |
| students | all (CRUD) | view, create, update: own | - |
| documents (lead and student) | all (CRUD) | view, create, update, delete: own | - |
| applications | all (CRUD) | view, create, update: own | - |
| catalog (countries to intakes, document types) | all (CRUD) | view | view |
| accounts | all (CRUD) | - | - |
| dashboard | all | view: own | - |

# 3. Auth

| Method | Path | Permission | Description |
|---|---|---|---|
| POST | `/auth/login` | public | Email and password; returns token and user |
| GET | `/auth/me` | any user | Current user with permissions |
| POST | `/auth/change-password` | any user | `{ currentPassword, newPassword }` |

### POST /auth/login

```
// request
{ "email": "counselor@example.com", "password": "secret" }

// 200 response
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": 3,
    "fullName": "Asha Rai",
    "email": "counselor@example.com",
    "role": "counselor",
    "permissions": [
      { "module": "leads", "action": "view", "scope": "own" },
      { "module": "catalog", "action": "view", "scope": "all" }
    ]
  }
}
```

The client uses `permissions` to show or hide menu items and buttons. The server still enforces every check.

## 4. Users and roles

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/users` | users:view | List users. Filters: `role`, `isActive`, `q` (name, email) |
| POST | `/users` | users:create | `{ fullName, email, password, roleId }` |
| GET | `/users/:id` | users:view | One user |
| PATCH | `/users/:id` | users:update | Update name, role, `isActive`, or reset `password` |
| GET | `/users/assignable` | any user | Active counselors as `[{ id, fullName }]`, for assignment dropdowns |
| GET | `/roles` | users:view | Roles with their permission matrix |

User object: `{ id, fullName, email, role, isActive, lastLoginAt, createdAt }`. Deactivating a user (`isActive: false`) keeps their records; there is no hard delete.

## 5. Leads

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/leads` | leads:view | List with filters (below) |
| POST | `/leads` | leads:create | Create a lead |
| GET | `/leads/:id` | leads:view | Lead detail |
| PATCH | `/leads/:id` | leads:update | Edit lead fields |
| PATCH | `/leads/:id/status` | leads:update | `{ status, lostReason? }`; logs a `status_change` activity |
| PATCH | `/leads/:id/assign` | leads:update | `{ assignedTo }` (counselor user ID) |
| POST | `/leads/:id/convert` | leads:update and students:create | Convert to a student |
| DELETE | `/leads/:id` | leads:delete | Hard delete (admin only) |

**GET /leads filters:** `status`, `source`, `assignedTo`, `destinationCountryId`, `intakeYear`, `createdFrom`, `createdTo`, `q` (name, email, phone). **Sort:** `createdAt`, `updatedAt`, `fullName`.

**Lead object**

```
{
  "id": 41,
  "fullName": "Sunita Thapa",
  "email": "sunita@example.com",
  "phone": "+977-9800000000",
  "source": "facebook",
  "destinationCountry": { "id": 2, "name": "Australia" },
  "desiredIntake": { "term": "fall", "year": 2027 },
  "testStatus": "preparing",
  "status": "contacted",
  "lostReason": null,
  "notes": "Interested in nursing",
  "assignedTo": { "id": 3, "fullName": "Asha Rai" },
  "createdBy": { "id": 5, "fullName": "Front Desk" },
  "convertedAt": null,
  "studentId": null,
  "createdAt": "2026-11-02T08:15:00Z",
  "updatedAt": "2026-11-03T10:00:00Z"
}
```


**POST /leads** requires `fullName` and at least one of `email` or `phone`. Optional: `source`, `destinationCountryId`, `desiredIntake`, `testStatus`, `notes`, `assignedTo`. Status starts as `new`. If a counselor creates a lead without `assignedTo`, it is assigned to them.

**Status rules:** any status can move to `lost` (requires `lostReason`). `converted` cannot be set directly; it is set only by the convert endpoint. A converted lead's status cannot change.

**POST /leads/:id/convert**

- Body is optional and can pre-fill student fields: `{ dateOfBirth?, nationality?, address?, guardianName?, guardianPhone? }`.
- Fails with `409` if the lead is already converted or is `lost`.
- In one transaction the server: creates the student (copying name, contact, assignee), sets the lead to `converted` with `convertedAt`, creates a checklist item for every document type flagged as default, and marks an item `received` (linking the file) when the lead already has an attachment of that type.
- Returns `201` with the student object.

### Lead activities (timeline)

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/leads/:id/activities` | leads:view | Timeline, newest first |
| POST | `/leads/:id/activities` | leads:update | `{ type, content, occurredAt? }`; type is `call`, `note`, `meeting` or `email` |

Activity: `{ id, type, content, occurredAt, createdBy: { id, fullName } }`. `status_change` entries are created by the server and cannot be posted.

### Lead documents

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/leads/:id/documents` | documents:view | Attachments on this lead |
| POST | `/leads/:id/documents` | documents:create | `multipart/form-data`: `file`, optional `documentTypeId`, optional `note` |
| DELETE | `/leads/:id/documents/:docId` | documents:delete | Remove the attachment |

Document: `{ id, documentType: { id, name } | null, note, file: { id, originalName, mimeType, sizeBytes }, createdBy, createdAt }`.

## 6. Tasks and follow-ups

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/tasks` | tasks:view | List with filters (below) |
| POST | `/tasks` | tasks:create | Create a task |
| PATCH | `/tasks/:id` | tasks:update | Edit title, description, due date, assignee |

| Method | Path | Permission | Description |
|---|---|---|---|
| POST | `/tasks/:id/complete` | tasks:update | Marks `done` and sets `completedAt` |
| DELETE | `/tasks/:id` | tasks:delete | Delete |

**GET /tasks filters:** `view` (`today`, `overdue`, `upcoming`, `all`; default `all`), `status`, `assignedTo`, `leadId`, `studentId`. **Sort:** `dueAt`, `createdAt`.

- `view=today`: open tasks due today. `view=overdue`: open tasks with `dueAt` in the past. `view=upcoming`: open tasks due after today.
- **POST body:** `{ title, description?, dueAt, leadId?, studentId?, assignedTo? }`. At least one of `leadId` or `studentId` is required. `assignedTo` defaults to the current user.
- Task: `{ id, title, description, dueAt, status, isOverdue, lead: { id, fullName } | null, student: { id, fullName } | null, assignedTo, createdBy, completedAt }`.

## 7. Students

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/students` | students:view | List. Filters: `assignedTo`, `preferredCountryId`, `q` (name, email, phone). Sort: `createdAt`, `fullName` |
| GET | `/students/:id` | students:view | Full profile (below) |
| PATCH | `/students/:id` | students:update | Edit personal fields, `preferredCourses`, `notes`, `assignedTo` |
| PUT | `/students/:id/preferred-countries` | students:update | `{ countryIds: [1, 2] }` replaces the list |

Students are created only through lead conversion, so there is no `POST /students`.

**GET /students/:id** returns the student with nested data:

```
{
  "id": 12,
  "leadId": 41,
  "fullName": "Sunita Thapa",
  "email": "sunita@example.com",
  "phone": "+977-9800000000",
  "dateOfBirth": "2003-04-12",
  "nationality": "Nepali",
  "preferredCourses": "Nursing, Public Health",
  "preferredCountries": [{ "id": 2, "name": "Australia" }],
  "assignedTo": { "id": 3, "fullName": "Asha Rai" },
  "education": [{ "id": 1, "level": "High School", "institution": "ABC
School", "endYear": 2021, "grade": "3.6 GPA" }],
  "testScores": [{ "id": 1, "testName": "IELTS", "overallScore": 6.5,
"sectionScores": { "listening": 7, "reading": 6.5, "writing": 6, "speaking":
6.5 }, "testDate": "2026-09-20" }],
  "createdAt": "2026-11-10T06:00:00Z"
}
```

## Education and test scores

Same pattern for both sub-resources; permissions are students:update (students:view for GET).

| Method | Path | Description |
|---|---|---|
| POST | `/students/:id/education` | `{ level, institution, fieldOfStudy?, startYear?, endYear?, grade? }` |
| PATCH | `/students/:id/education/:eid` | Edit |
| DELETE | `/students/:id/education/:eid` | Remove |
| POST | `/students/:id/test-scores` | `{ testName, overallScore?, sectionScores?, testDate? }`; testName is `IELTS`, `TOEFL`, `PTE`, `Duolingo`, `SAT`, `ACT`, `GRE`, `GMAT` or `Other` |
| PATCH | `/students/:id/test-scores/:sid` | Edit |
| DELETE | `/students/:id/test-scores/:sid` | Remove |

## Document checklist

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/students/:id/documents` | documents:view | Checklist items plus the original lead attachments |
| POST | `/students/:id/documents` | documents:create | Add a checklist item: `{ documentTypeId }`; `409` if it already exists |
| PATCH | `/students/:id/documents/:itemId` | documents:update | `{ status?, remarks? }` |
| POST | `/students/:id/documents/:itemId/file` | documents:create | `multipart/form-data` with `file`; replaces any previous file and sets status to `received` if it was `pending` |
| DELETE | `/students/:id/documents/:itemId` | documents:delete | Remove the checklist item |

### GET response

```
{
  "checklist": [
    {
      "id": 7,
      "documentType": { "id": 1, "name": "Passport" },
      "status": "verified",
      "file": { "id": 55, "originalName": "passport.pdf", "mimeType":
"application/pdf", "sizeBytes": 482113 },
      "remarks": null,
      "verifiedBy": { "id": 3, "fullName": "Asha Rai" },
      "verifiedAt": "2026-11-11T09:00:00Z"
    }
  ],
  "leadAttachments": [ /* same shape as lead documents */ ]
}
```

Status moves `pending` → `received` → `verified`. Setting `verified` records `verifiedBy` and `verifiedAt`. A hard copy can be marked `received` without a file.

### Files

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/files/:id` | access to the owning lead or student document | Streams the file; `?download=1` forces download |

Uploads: max 10 MB; PDF, JPG, PNG, DOCX only. Files are stored outside the web root and are only served through this authenticated route.

## 8. Catalog

Countries, universities, programs, intakes, and document types. All `GET` routes need `catalog:view`; create and update need `catalog:create` and `catalog:update` (admin). All lists return only active items unless `?includeInactive=true` (admin).

The cascading filter (country, university, program, intake) is built from the parent filters below: pick a country, load its universities, and so on.

| Method | Path | Filters | Body (create) |
|---|---|---|---|
| GET, POST | `/countries` | `q` | `{ name, isoCode? }` |
| PATCH | `/countries/:id` | | `{ name?, isoCode?, isActive? }` |
| GET, POST | `/universities` | `countryId`, `q` | `{ countryId, name, city?, website?, notes? }` |
| PATCH | `/universities/:id` | | any field, plus `isActive` |
| GET, POST | `/programs` | `universityId`, `countryId`, `level`, `q` | `{ universityId, name, level, fieldOfStudy?, durationMonths?, tuitionFee?, currency? }` |
| PATCH | `/programs/:id` | | any field, plus `isActive` |
| GET, POST | `/intakes` | `programId`, `year` | `{ programId, term, year, applicationDeadline? }` |
| PATCH | `/intakes/:id` | | any field, plus `isActive` |
| GET, POST | `/document-types` | | `{ name, inDefaultChecklist? }` |
| PATCH | `/document-types/:id` | | `{ name?, inDefaultChecklist?, isActive? }` |

`level` is `foundation`, `diploma`, `bachelor`, `master`, `phd` or `other`. `term` is `spring`, `summer`, `fall` or `winter`.

Programs and intakes embed their parents in responses so a list row is self-contained:

```
{
  "id": 9,
  "term": "fall",
  "year": 2027,
  "applicationDeadline": "2027-03-31",
  "program": { "id": 4, "name": "MSc Data Science", "level": "master" },
  "university": { "id": 2, "name": "University of Sydney" },
  "country": { "id": 2, "name": "Australia" }
}
```

# 9. Applications

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/applications` | applications:view | List with filters (below) |
| POST | `/applications` | applications:create | `{ studentId, programIntakeId, deadline?, notes? }` |
| GET | `/applications/:id` | applications:view | Detail with status history |
| PATCH | `/applications/:id` | applications:update | Edit `deadline`, `notes` |
| PATCH | `/applications/:id/status` | applications:update | `{ status, note? }` |
| DELETE | `/applications/:id` | applications:delete | Admin only |

**GET /applications filters:** `studentId`, `status`, `countryId`, `universityId`, `programId`, `intakeId`, `assignedTo`, `deadlineFrom`, `deadlineTo`. **Sort:** `deadline`, `createdAt`.

### Application object

```
{
  "id": 20,
  "student": { "id": 12, "fullName": "Sunita Thapa" },
  "intake": { "id": 9, "term": "fall", "year": 2027 },
  "program": { "id": 4, "name": "MSc Data Science", "level": "master" },
  "university": { "id": 2, "name": "University of Sydney" },
  "country": { "id": 2, "name": "Australia" },
  "status": "submitted",
  "deadline": "2027-03-31",
  "submittedAt": "2026-12-01T05:00:00Z",
  "notes": null,
  "history": [
    { "fromStatus": null, "toStatus": "draft", "changedBy": "Asha Rai",
"changedAt": "2026-11-20T07:00:00Z", "note": null },
    { "fromStatus": "draft", "toStatus": "submitted", "changedBy": "Asha
Rai", "changedAt": "2026-12-01T05:00:00Z", "note": "All documents verified" }
  ]
}
```

- `deadline` defaults to the intake's `applicationDeadline` when omitted. The `history` array is only on the detail route.
- A student can apply to a given intake once; a duplicate returns `409`.
- **Allowed status transitions:** `draft` → `submitted`; `submitted` → `offer` or `rejected`; `offer` → `accepted` or `rejected`. Anything else returns `409`. Moving to `submitted` sets `submittedAt`. Every change adds a history row.

## 10. Dashboard

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/dashboard/summary` | dashboard:view | All dashboard widgets in one response |

Query: `month=2026-11` (defaults to the current month). Admin sees all data; a counselor sees only their own leads, tasks and applications.

```
{
  "month": "2026-11",
  "leadsByStatus": [{ "status": "new", "count": 14 }, { "status":
"contacted", "count": 9 }],
  "leadsBySource": [{ "source": "facebook", "count": 11 }, { "source":
"walk_in", "count": 6 }],
  "newLeadsThisMonth": 23,
  "conversionsThisMonth": 5,
  "tasksDueToday": 4,
  "tasksOverdue": 2,
  "applicationsByStatus": [{ "status": "draft", "count": 8 }, { "status":
"submitted", "count": 12 }],
  "upcomingDeadlines": [
    { "applicationId": 20, "studentName": "Sunita Thapa", "university":
"University of Sydney", "program": "MSc Data Science", "deadline": "2027-03-
31" }
  ]
}
```

`upcomingDeadlines` lists applications still in `draft` or `submitted` with a deadline in the next 30 days, soonest first, limit 10. `leadsByStatus` and `leadsBySource` cover all leads in scope; `conversionsThisMonth` counts leads whose `convertedAt` falls in the month.

## 11. Accounts (admin only)

Internal office bookkeeping: rent, salaries, utilities, and similar. It does not track student fees, invoices or commissions.

| Method | Path | Permission | Description |
|---|---|---|---|
| GET | `/accounts/categories` | accounts:view | Categories with `kind` (`income` or `expense`) |
| POST | `/accounts/categories` | accounts:create | `{ name, kind }` |
| PATCH | `/accounts/categories/:id` | accounts:update | `{ name?, isActive? }` |
| GET | `/accounts/entries` | accounts:view | Filters: `from`, `to`, `categoryId`, `kind`, `q` (description, payee). Sort: `entryDate`, `amount` |

| Method | Path | Permission | Description |
|---|---|---|---|
| POST | `/accounts/entries` | accounts:create | `{ categoryId, entryDate, amount, description?, payeeName?, periodMonth?, referenceNo? }` |
| PATCH | `/accounts/entries/:id` | accounts:update | Edit any field |
| DELETE | `/accounts/entries/:id` | accounts:delete | Delete |
| GET | `/accounts/summary` | accounts:view | Monthly totals |

- `amount` is a positive number; whether it is money in or out comes from the category's `kind`.
- `periodMonth` is the first day of the month an entry covers (for example `2026-11-01` for November rent or salary).
- **GET /accounts/summary?month=2026-11** returns `{ month, totalIncome, totalExpense, net, byCategory: [{ categoryId, name, kind, total }] }`.
- The app uses one currency throughout; it is a display setting on the client.

## 12. Assumptions to confirm

- Front desk can only see leads they created, not every lead. If receptionists need to check for duplicates across all leads, change their `leads:view` scope to `all` in the seed data.
- Lead documents are stored per lead and shown on the student profile after conversion (via `leadAttachments`) rather than copied, so there is one copy of each file.
- Roles and permissions are fixed by the seed data in this version; there is no UI for editing the matrix.
- Lead status `converted` is only reachable through the convert endpoint, so a lead cannot be marked converted without a student record existing.